

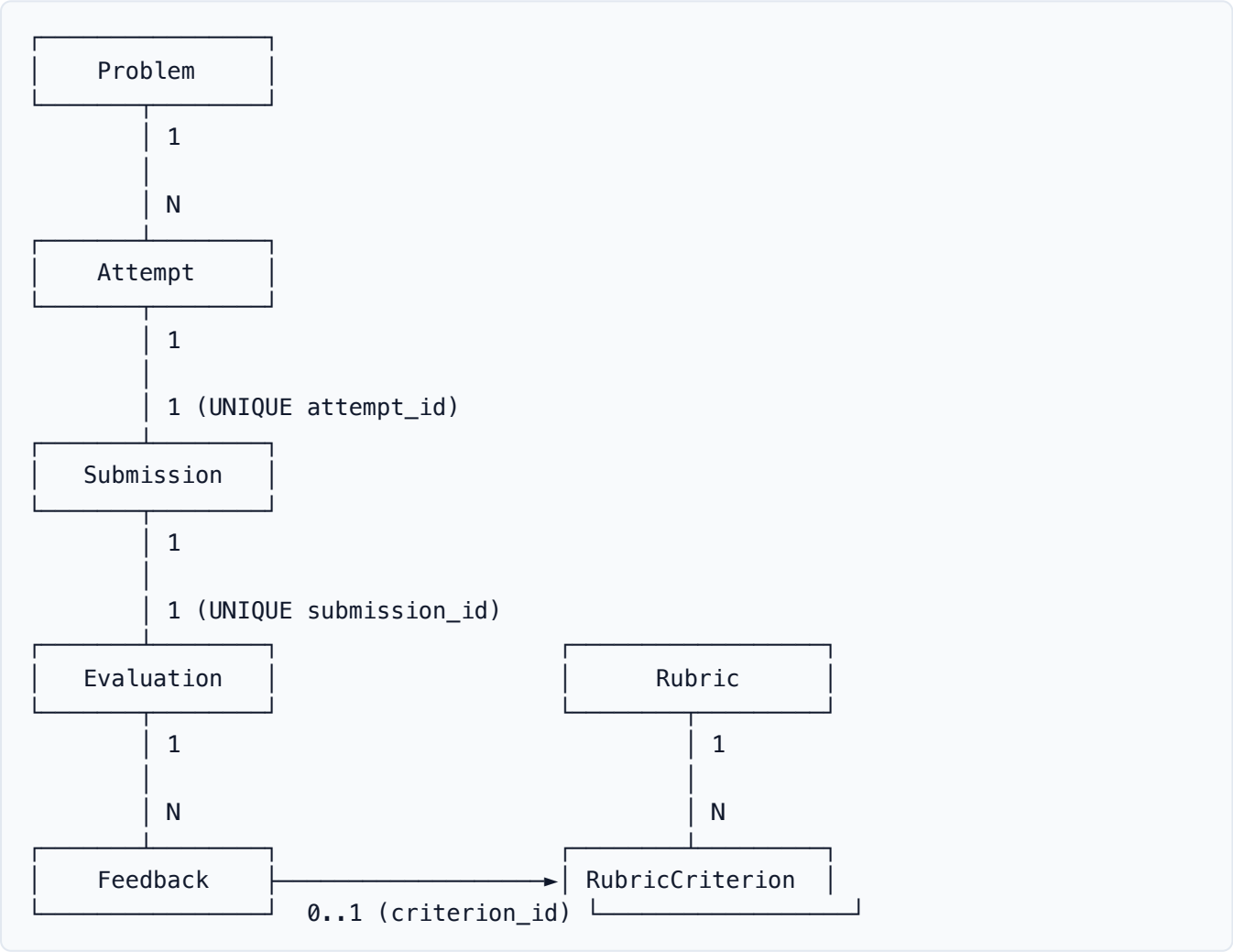
System Architecture & Low-Level Design Document

1. Executive Summary

The **LLD Practice Platform** is a focused educational platform designed to help software engineering learners practice Low-Level Design iteratively. It evaluates student-submitted software designs against a standardized, 100-point rubric and provides evidence-backed, explainable feedback.

This prototype demonstrates disciplined domain modeling, strict database constraints, clean state transitions, and a resilient evaluation architecture within a simple, maintainable modular monolith.

2. Core Domain Model



Entity Responsibilities

Entity	Primary Responsibility	Invariants & Constraints
Problem	Defines the LLD challenge requirements, constraints, and scope.	<code>slug</code> is unique. Seeded with 4 canonical problems. Protected from accidental deletion (<code>onDelete: Restrict</code>).
Attempt	Represents one distinct learner practice session on a problem.	Belongs to a problem. Preserves chronological history. Unlimited attempts allowed per problem. Starts in <code>IN_PROGRESS</code> .
Submission	Represents the actual design artifact submitted by the learner.	Strictly 1:1 with <code>Attempt</code> . Immutable upon submission. Holds content type (<code>TEXT</code> , extensible to <code>DIAGRAM</code> , <code>CODE</code>). Saved before evaluation begins.
Evaluation	Represents the evaluation process, total score, and lifecycle status.	Strictly 1:1 with <code>Submission</code> via <code>@unique</code> on <code>submission_id</code> . Prevents duplicate evaluation records.
Feedback	Contains structured assessment for a specific rubric dimension.	Cascades from <code>Evaluation</code> . Captures <code>evidence</code> , <code>concern</code> , <code>suggestion</code> , <code>score</code> , and <code>maxScore</code> . Atomically persisted.
Rubric	Reusable collection of criteria summing to a fixed score target (100 pts).	<code>name</code> is unique. <code>DEFAULT_LLD_RUBRIC</code> is flagged as default.
RubricCriterion	Specific evaluation dimension within a rubric.	Unique name per rubric. Contains <code>maxScore</code> and <code>orderIndex</code> .

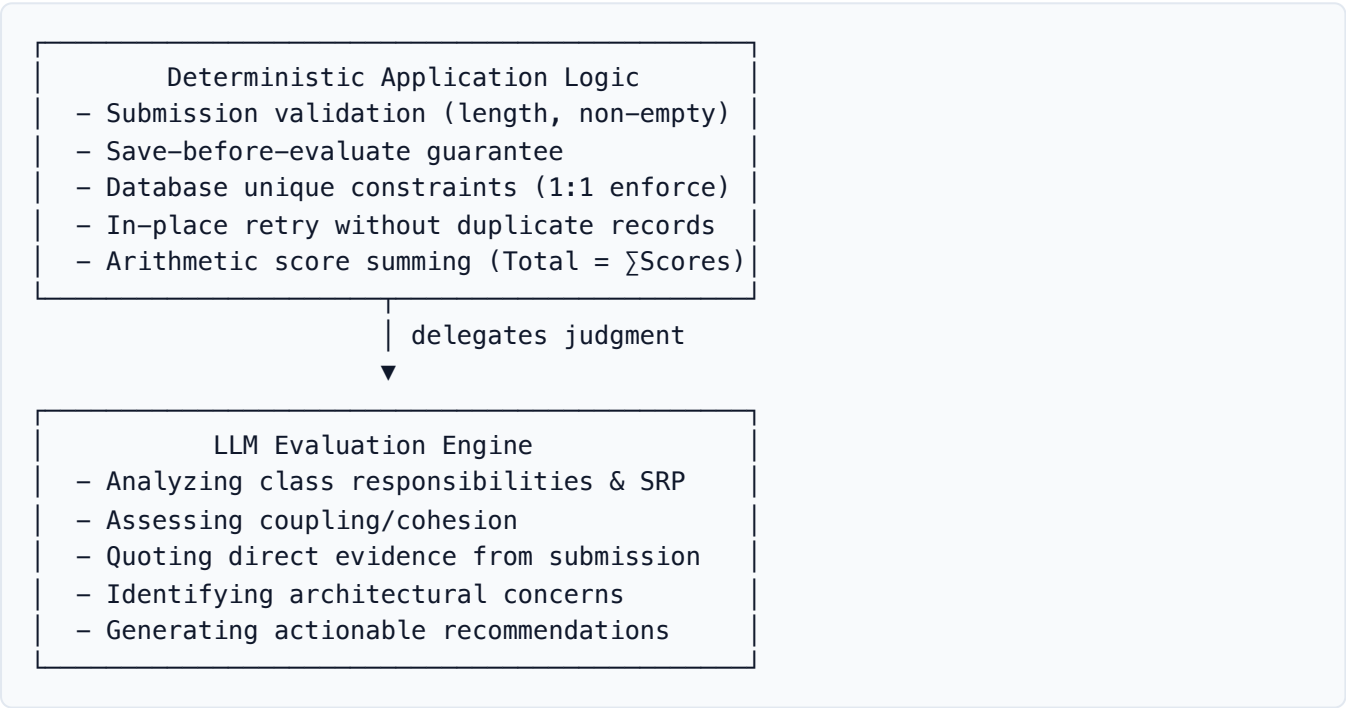
3. Evaluation Architecture & Rubric Design

Standard 100-Point Evaluation Rubric

#	Dimension	Max Pts	Focus Area
1	Requirement Understanding	15	Identifying core functional requirements, scope boundaries, and assumptions.
2	Class Responsibilities	20	Single Responsibility Principle (SRP); avoidance of god-objects.
3	Encapsulation & Interfaces	15	Clean public contracts, minimal interfaces, state protection.
4	Coupling & Cohesion	15	Loose coupling between subsystems; high internal cohesion; Dependency Inversion.

#	Dimension	Max Pts	Focus Area
5	Abstraction / Design Patterns	10	Appropriate, deliberate pattern selection (Strategy, State, Factory, Observer).
6	Extensibility	15	Open-Closed Principle (OCP); ease of adding new rules/types without rewriting core logic.
7	Edge Cases & Testability	10	Concurrency handling, failure modes, validation guards, and unit testability.
Total		100	

Deterministic vs. AI Responsibilities





Allowed State Transitions

- PENDING → EVALUATING
- EVALUATING → COMPLETED
- EVALUATING → FAILED
- FAILED → EVALUATING

All other transitions (e.g. PENDING → COMPLETED, COMPLETED → EVALUATING, FAILED → COMPLETED) are rejected with 400 Bad Request.

The "Save-Before-Evaluate" Principle

A common failure mode in naive AI applications is: Submit → Call LLM → (if LLM fails, everything is lost).

Our architecture guarantees:

1. **Submission Persisted First:** The learner's text is saved to PostgreSQL and the attempt is marked completed.
2. **Evaluation Initialized:** An Evaluation record is created in PENDING or EVALUATING status.
3. **Evaluator Execution:** The LLM is called with structured JSON output schema and grounded prompts.
4. **On Success:**
 - Total score is calculated deterministically from criterion scores.
 - Feedback records are saved in an atomic transaction.
 - Evaluation status transitions to COMPLETED.
5. **On Failure:**
 - Evaluation status transitions to FAILED with errorMessage.
 - **The student's submission is 100% safe.**
 - Retrying the evaluation executes an in-place UPDATE on the existing evaluation record (FAILED → EVALUATING → COMPLETED), maintaining database integrity and avoiding duplicate rows.

Learner Retry vs. Evaluation Retry

- **Learner Retry ("Try Again"):** Creates a completely **NEW** Attempt record. The previous attempt, submission, evaluation, and feedback remain 100% intact for comparative analysis.
- **Evaluation Retry:** Used when an evaluation fails due to network/LLM timeouts. It reuses the **SAME** Evaluation record, transitioning it back to `EVALUATING` and re-running the evaluator without duplicating attempts or submissions.

5. Architectural Change Tests

Change Test A: Moving from Text Submissions to UML/Class Diagrams

- **Current implementation:** `Submission.type = "TEXT"` with text content.
- **Hypothetical Change:** Candidate uploads a Mermaid diagram or draws a class diagram.
- **Impact:**
 - `Submission.type` accepts `"DIAGRAM"`.
 - `Submission.content` stores diagram DSL (Mermaid/JSON) or an S3 asset URL.
 - `Problem`, `Attempt`, `Evaluation`, and `Feedback` models **remain 100% unchanged**.

Change Test B: Adding Rule-Based or Human Reviewers

- **Current implementation:** `Evaluator` interface implemented by `LLMEvaluator`.
- **Hypothetical Change:** Introduce automated static analysis (`RuleBasedEvaluator`) or mentor reviews (`HumanEvaluator`).
- **Impact:**

```
export class RuleBasedEvaluator implements Evaluator { ... }  
export class HumanEvaluator implements Evaluator { ... }
```

The domain service (`EvaluationService`) accepts any class satisfying the `Evaluator` contract. The database stores `evaluatorType = "RULE_BASED"` or `"HUMAN"`. The feedback schema remains identical.

6. Future Scaling Considerations (Architecture Design Note)

While the MVP is built as a clean monolith, the domain boundaries are designed to scale cleanly:

1. Asynchronous Evaluation Worker Queue (Redis)

- **Problem:** Large language models can take 5–15 seconds to return detailed structured feedback. Synchronous HTTP request-response can encounter browser timeouts under heavy load.
- **Evolution:**
 - The `POST /api/submissions/:id/evaluate` endpoint immediately creates the `Evaluation` row in `PENDING` state and enqueues a job in a **Redis-backed queue** (e.g. BullMQ).
 - A pool of background evaluation worker processes consumes jobs, invokes the `Evaluator`, and updates the database.
 - The client polls `GET /api/evaluations/:submissionId` or receives an SSE/WebSocket notification when `COMPLETED`.

2. Multi-Consumer Event Streaming (Kafka)

- **Problem:** When an evaluation completes, multiple downstream services may need to act on that event (e.g. Learner Analytics, Mentor Alerts, Badge & Gamification Engine, Training Data Pipeline).
- **Evolution:**
 - Upon completing an evaluation, the application publishes an `EvaluationCompletedEvent` to an **Apache Kafka topic** (`lld.evaluations.completed`).
 - Downstream microservices subscribe independently without adding coupling or latency to the core practice platform.

7. Key Trade-Offs Made

1. **Monolith over Microservices:** Selected to maximize development velocity, domain coherence, and operational simplicity.
2. **Text-First Submissions over UML Canvas:** Text design captures the core cognitive requirements of LLD (responsibilities, interfaces, trade-offs) while keeping focus squarely on design reasoning rather than canvas diagram construction.
3. **Deterministic Score Summing:** Rather than trusting the LLM to output a total score, the backend calculates $\sum \text{criterionScores}$, eliminating arithmetic hallucinations.
4. **In-place Evaluation Retries:** Rather than creating orphaned evaluation records, retries update the existing unique record, preventing database bloat.